ARTIFICIAL INTELLIGENCE

TOPIC 5.3

Foundation models

Explanatory study notes

Theory, worked examples and revision questions

What you should be able to do

Explain broad pretraining and downstream adaptation; distinguish self-supervision, supervised fine-tuning and reinforcement learning; interpret total versus active parameters; compare BERT-style, GPT-style, CLIP, DINOv2 and SAM models; identify what a model does and what its output means.

Based on the teacher PDFs supplied in `Lecture Slides`. The notes explain and reorganize that material. Worked calculations are study examples derived from the slide concepts, unless explicitly identified as a slide exercise. Clarifications of ambiguous or incorrect expressions are marked in context. References use **PDF page numbers**, counted from 1, rather than printed slide numbers.

English edition • 24 October 2026

Contents

1 What makes a model a foundation model? 3

2 Architecture, objective and adaptation 3

2.1 Ways to reuse a pretrained model . . . 3

3 Where the training signal comes from 4

3.1 SFT versus preference optimization . . . 4

3.2 Verifiable rewards . . . 5

3.3 A simple reward calculation . . . 5

4 Scale, storage and mixture of experts 5

5 CLIP: align images and text 6

5.1 Zero-shot classification . . . 6

6 DINOv2 and SAM 6

6.1 DINOv2: reusable visual features . . . 6

6.2 SAM: promptable segmentation . . . 7

7 Language-conditioned image generation 7

8 Benefits and limitations to explain 7

9 Revision questions and answers 8

9.1 Questions . . . 8

9.2 Answers . . . 8

10 Source guide 9

1 What makes a model a foundation model?

Teacher slides: 5.3. Foundation models, pp. 9–13.

A foundation model is trained on broad data at scale and can be adapted to many downstream tasks. The key idea is **transfer learning at scale**: an expensive shared pretraining stage produces reusable representations or behaviors, instead of training a separate model from scratch for every task.

A transformer is an architecture; a foundation model describes the scope of training and reuse. The terms are not synonyms. A small transformer trained only for one narrow dataset is not automatically a foundation model. Likewise, a foundation model can be useful as an encoder without being a conversational or image-generation system.

Broad pretraining can provide useful features, language patterns and cross-modal associations. Performance still depends on the downstream task, domain, data quality and adaptation. Broad training does not guarantee accurate answers on every new problem.

Identify three things

For any named model, ask: what input does it accept, what objective was used to train it, and what output does it produce? A feature vector, a token distribution, a mask and a generated image are different outputs, even if the models share transformer components.

2 Architecture, objective and adaptation

Teacher slides: 5.3. Foundation models, pp. 5–8, 15–17.

Family	Training/attention idea	Useful output
BERT-style encoder	Bidirectional input context; predict hidden tokens. Original BERT also used next-sentence prediction.	Contextual token features or a sequence representation for a task head.
GPT-style decoder	Causal self-attention; predict the next token from the prefix.	Next-token probabilities, used repeatedly for generation.
Encoder–decoder	Encode a source and generate a target with causal self-attention plus cross-attention.	A conditional sequence, such as a translation or transcription.

BERT-style masked-token prediction learns from both sides of a missing token. GPT-style next-token prediction cannot use the unknown continuation. Both can obtain training targets from text itself, even though their information restrictions differ.

The slides use named model families and historical parameter counts to illustrate these ideas. Treat those as course examples, not a specification of every version or a ranking of current products. Detailed internals of some proprietary models are not publicly established.

2.1 Ways to reuse a pretrained model

Method	What changes?
Frozen feature extraction	Use the representation as input to a new task head; keep the backbone fixed.
Fine-tuning	Update some or all pretrained parameters on the downstream training objective.
Prompting	Supply instructions or context as input; ordinary inference leaves weights fixed.
Few-shot prompting	Include several task examples in the prompt; the model conditions on them without a gradient update.
Zero-shot use	Apply the model to a task/category without task-specific example training in the stated setup.

The meaning of zero-shot depends on the evaluation protocol. It does not mean the pretraining data necessarily contained nothing related to the category. In-context examples change the current computation and context, not ordinarily the stored parameters.

3 Where the training signal comes from

Teacher slides: 5.3. Foundation models, pp. 13–14; 5.4. Unsupervised learning, pp. 2, 24.

The slide pipeline separates several sources of supervision:

Stage	Training signal	Interpretation
Self-supervised pretraining	Targets constructed from observed data, such as held-out or next tokens.	Learn broad structure without manually labeling every prediction target.
Supervised fine-tuning (SFT)	Demonstrated desired responses or labeled task examples.	Learn to imitate appropriate outputs for prompts/tasks.
Preference-based RL	Preferences used to train or supply a reward signal.	Sample responses and adjust behavior to favor rewarded ones.
Verifiable RL	Automatic checkers, such as answer matching or tests.	Reward outputs according to an externally evaluated criterion.

These are complementary mechanisms. A vision-language model (VLM) can combine image/text pretraining with instruction examples and later reward-based training. One model's complete training history need not fit exactly one supervision label.

3.1 SFT versus preference optimization

SFT treats a desired response as a target sequence and increases its likelihood. A preference comparison says that one candidate response is better than another; it need not provide a perfect response token by token. In a common RLHF setup, a reward model learns such preferences and an optimization procedure increases expected reward.

Not every preference-based method explicitly runs an RL rollout-and-update loop. The conceptual point is the difference between imitating a demonstrated target and optimizing a preference-derived signal, not that every named post-training recipe has identical machinery.

3.2 Verifiable rewards

A checker may determine whether a mathematical answer matches a reference, whether generated code passes tests, or how well a predicted box overlaps a target. These signals can provide scalable feedback. The reward only measures what the checker actually checks; passing incomplete tests does not establish correctness in every possible case.

Alignment is not a guarantee of factual correctness

The slides mention reducing hallucinations through post-training. Reward optimization can improve measured behavior, but a high preference score or fluent answer does not prove truth. A learned reward can reflect annotator biases, and an automatic verifier can be incomplete or flawed.

3.3 A simple reward calculation

Suppose a policy chooses response A with probability 0.7 and response B with probability 0.3, and a checker assigns rewards 1 and 0. Expected reward is $0.7(1) + 0.3(0) = 0.7$. Increasing A's probability to 0.9 raises it to 0.9. This is a toy illustration of expected reward, not a complete policy-gradient update.

4 Scale, storage and mixture of experts

Teacher slides: 5.3. Foundation models, pp. 12, 17, 26.

Parameter count is the number of learned scalar coefficients. Storage also depends on precision. For raw weights only,

$$\text{bytes} = (\text{parameter count}) \times \frac{\text{bits per stored parameter}}{8}.$$

A hypothetical 100-million-parameter model needs about 400 MB at 32 bits or 200 MB at 16 bits, using decimal MB. These estimates exclude metadata, activations, gradients, optimizer state and inference caches.

In a sparse mixture-of-experts (MoE) model, a router selects a subset of expert subnetworks for each token. Some components are shared and used for every token. Distinguish:

- **Total parameters**: all shared and expert parameters stored by the full model.
- **Active parameters**: the subset used along a given token's computation.

A toy model with 40 million shared parameters and 16 experts of 20 million each has 360 million total parameters. If two experts are used per token, that token uses $40 + 2(20) = 80$ million parameters, about 22.2% of the total.

This ratio is not a guaranteed speedup. Routing, communication, memory access and the operations in the active path also matter. Different tokens can select different experts, so the full set may be used across a batch. Sparse activation reduces some computation without making the unused experts disappear from model storage.

5 CLIP: align images and text

Teacher slides: 5.3. Foundation models, pp. 19–20.

CLIP uses an image encoder and a text encoder to map paired images and descriptions into a shared representation space. Training increases the similarity of matching image–text pairs relative to mismatched pairs.

With B paired examples in a batch, the similarity matrix has $B \times B$ entries. The diagonal represents matched pairs under aligned ordering; the off-diagonal entries provide mismatches. Both image-to-text and text-to-image matching can provide training signals.

5.1 Zero-shot classification

To classify an image among candidate categories:

1. Write a text description for each candidate category.
2. Encode the image and the candidate texts.
3. Compare image/text embeddings, often after normalization.
4. Select the category whose description has the largest similarity.

This supplies a classification rule without fitting a new labeled classifier for those categories. Performance can depend on wording and which candidate labels are offered.

For unit-normalized vectors, cosine similarity is their dot product. If the image vector is $(1, 0)$ and candidate vectors are $(0.8, 0.6)$ and $(0, 1)$, similarities are 0.8 and 0. The first is the better match among these candidates. A similarity of 0.8 is not automatically an 80% calibrated probability of correctness.

An embedding model is not an image generator

CLIP supplies aligned representations and similarity scores. It can guide or provide conditioning/evaluation in a larger generative system, but the image encoder/text encoder pair alone does not decode a new image from a prompt.

6 DINOv2 and SAM

Teacher slides: 5.3. Foundation models, pp. 21–22.

6.1 DINOv2: reusable visual features

DINOv2 is a self-supervised visual-representation example. A student learns from a teacher using different views of images without requiring semantic class labels or text captions for the pretraining objective. Image-level and patch-level information support reusable visual features.

The teacher is part of the training mechanism, not a human annotator supplying class names. Learned features can then support image or pixel tasks with an appropriate readout. The slide's broad transfer claim should not be interpreted as a guarantee that every task works perfectly without adaptation or a task-specific head.

6.2 SAM: promptable segmentation

The Segment Anything Model illustrates a foundation model for segmentation. It combines an image with a prompt specifying what to segment, such as points or a box in the original setting, and produces mask predictions.

A segmentation mask identifies image regions. It is not the same output as an image-classification label or a newly generated photograph. Promptable segmentation still requires deciding what object/region to request and evaluating whether the predicted boundary is useful. The exam-level distinction is the task and output; version-specific feature lists are not needed to explain that mechanism.

Example	Learned capability	Main output/use
CLIP	Image–text alignment from paired data.	Similarity/retrieval and zero-shot classification.
DINOv2	Visual representation from self-supervised image views.	Features for downstream image/pixel tasks.
SAM	Prompt-conditioned segmentation.	Object/region masks.

7 Language-conditioned image generation

Teacher slides: 5.3. Foundation models, pp. 23–25.

A text-to-image system can combine distinct components:

$$\text{text prompt} \longrightarrow \text{text features} \longrightarrow \text{conditioned denoising} \longrightarrow \text{image decoding}.$$

In a latent-diffusion example, a text encoder represents the prompt, a denoiser iteratively transforms a noisy latent while conditioning on the prompt, and a decoder maps the final latent to an image. The denoiser may use a U-Net or a transformer, depending on the model.

Next-token language prediction and continuous denoising are different objectives. Using language features to condition diffusion does not mean the image generator is simply selecting the next text token. Conversely, not every generative model must use diffusion. Topic 5.5 explains the broader distinctions.

8 Benefits and limitations to explain

Teacher slides: 5.3. Foundation models, pp. 10–14, 26.

Reusable representations can reduce task-specific training effort, allow transfer with fewer labels, and support multiple modalities. Large-scale training and deployment can require substantial memory, computation and data preparation.

Common pretrained components can also propagate biases and failure patterns across many downstream systems. Evaluate the actual task, relevant subgroups and deployment distribution. Explainability concerns ask what evidence a prediction used; fairness concerns ask how performance and impacts are distributed. Neither is settled solely by a model's size or overall accuracy.

These are conceptual issues raised by the course, not a legal checklist. For a short exam answer, connect a concrete property to a consequence: for example, a generic pretraining dataset may poorly represent a specialized engineering domain, so downstream validation is still necessary.

9 Revision questions and answers

9.1 Questions

1. Is every transformer a foundation model?
2. What distinguishes pretraining from downstream fine-tuning?
3. Does ordinary few-shot prompting update the model's weights?
4. Where do next-token pretraining targets come from?
5. How does a preference ranking differ from an SFT demonstration?
6. What does verifiable RL use as its reward source?
7. Can a model have many more total than active parameters?
8. What are CLIP's inputs and what does its similarity compare?
9. Is DINOv2's teacher necessarily a human supplying class labels?
10. What does SAM produce?
11. Does a text encoder alone implement diffusion image generation?
12. How many decimal MB store 50 million weights at 16 bits, excluding overhead?

9.2 Answers

1. No. Transformer is an architecture; foundation-model status concerns broad training and adaptability.
2. Pretraining learns a broadly reusable base; fine-tuning adapts parameters to a later task or behavior.
3. No. It supplies examples as input context.
4. The observed sequence itself: the following token is the target.
5. A ranking compares candidate outputs; SFT supplies a desired output to imitate.
6. An automatic checker or other verifiable evaluation criterion.
7. Yes, as in sparse MoE routing; all experts remain part of the total model.
8. Images and text; similarity compares their embeddings in a shared space.
9. No. It is a model within a self-supervised teacher–student procedure.
10. Prompt-conditioned segmentation masks.
11. No. A denoising/generative component and, in latent systems, an image decoder are also needed.
12. $50 \times 10^6 \times 2 = 100 \times 10^6$ bytes, or 100 MB.

10 Source guide

`5.3. Foundation models.pdf`: transformer-family recap (pp. 5–8), definition/scale and training signals (pp. 9–14), language/multimodal examples (pp. 15–18), CLIP/DINOv2/SAM (pp. 19–22), image generation (pp. 23–25), and efficiency/explainability/fairness questions (p. 26). Administrative pages are not study theory. Numerical storage, reward and MoE examples here are explanatory applications rather than specifications of named commercial systems.